

数据库工程作业

要求:

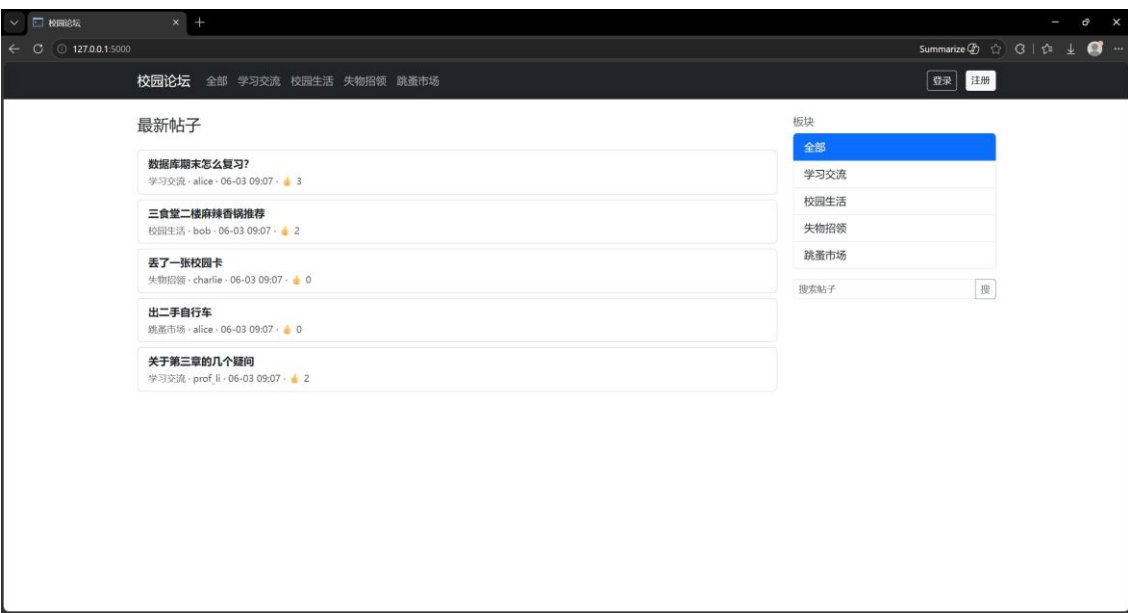
- 1. 完成一个小型的数据库信息管理系统（或部分功能），并填写工程作业报告；程序和报告请在规定时间之内上传。
- 2. 开发模式（B/S 或 C/S）、开发高级语言任选，后台数据库使用大型数据库管理系统（SQL Server、Oracle、MySQL 等），不要使用桌面数据库。
- 3. 报告中所列举的四种操作，每种操作举一个例子即可。
- 4. 作业成绩按照报告中的标准评分，程序只实现报告中涉及的部分即可。
- 5. 作业完成后，请将工程作业报告和程序打包提交给助教老师，并联系助教老师进行系统说明和演示，回答相关问题。

工程作业报告

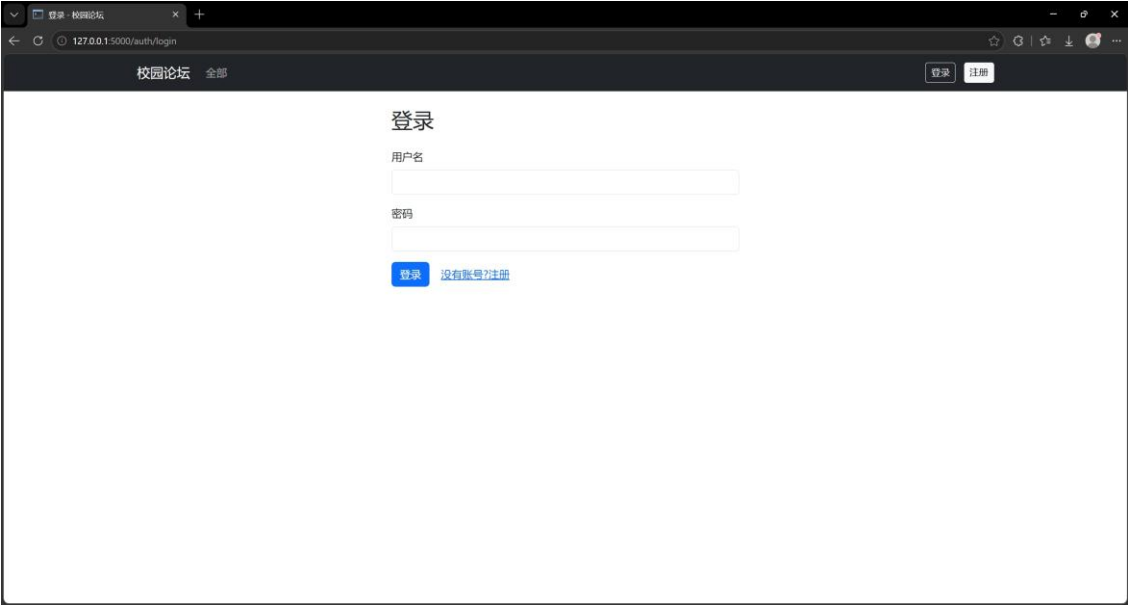
1. 项目信息（10 分）

学号	2410668	姓名	吕昊远	专业	计算机科学与技术
项目名称	在线校园论坛系统				
必备环境	Mysql、Python、HTML				
系统主要功能简介（4分）	本系统是一个基于 B/S 架构的校园论坛，采用 Flask 框架与原生 SQL（pymysql）开发，后台数据库使用 MySQL。主要功能包括： （1）用户管理：注册时区分学生/老师身份，密码经过哈希加密后存储；支持登录、登出与会话管理。 （2）板块管理：设置多个讨论板块，可按板块筛选帖子。 （3）帖子管理：用户可发布、浏览帖子，支持按标题或内容关键字进行检索，以及对帖子的点赞、评论与收藏操作。 （4）评论管理：支持对帖子评论，以及对评论进行回复。 （5）收藏管理：用户可创建多个收藏夹，将帖子收藏至指定收藏夹，并显示哪些收藏夹收藏了该帖子。 此外，系统提供终端管理脚本 admin.py 来模拟数据库管理员，可以集中演示视图查询、事务删除、存储过程注销等数据库高级操作。				

系统主要页面截图（6分）



（论坛首页）



（登陆界面）

丢了-一张校园卡 - 校园论坛

127.0.0.1:5000/posts/3

你好,kbs 登出 我的收藏

丢了一张校园卡

失物招领 · charlie · 2026-06-03 09:07 ·

0 点赞 举报

在图书馆 3 楼丢的,有拾到的麻烦联系

评论 (1)

kbs · 06-04 19:50

我找到了

kbs 111

回复

发表评论

写下你的想法...

提交

(帖子界面)

我的收藏 - 校园论坛

127.0.0.1:5000/user/profile

你好,kbs 登出 我的收藏

我的收藏

1 1 篇 查看

222 2 篇 qqz 查看

这里是收藏夹的名称 0 篇 还有备注 查看

新建收藏夹

收藏夹名称

备注(可选)

创建

2. 系统配置 (10 分)

说明		(2 分) 请说明系统配置情况 (后台数据库, 高级语言); (8 分) 请使用连接串连接高级语言和数据库, 并分析字符串的各个部分。			
配置 步骤 2 分	DBMS	1. MySQL 8.0			
	高级 语言	1. Python (Flask)			
		2. HTML			
连接串 分析 (6 分)		序 号	名称	功能说明	取值
		1	host	数据库服务器 ip 地址	localhost
		2	port	MySQL 服务监听端口	3306

	3	user	登录数据库的用户名	root
	4	password	登录用户的密码	123456hh
	5	database	要连接的数据库名	campus_forum
	6	charset	客户端上显示的字符集	utf8mb4
	7	cursorclass	游标类型	DictCursor
连接串代码 (截屏) (2 分)	项目使用 .env 配置文件 + pymysql 的参数字典方式连接数据库 (config.py): <pre> class Config: DB_HOST = os.getenv('DB_HOST', 'localhost') DB_PORT = int(os.getenv('DB_PORT', 3306)) DB_USER = os.getenv('DB_USER') DB_PASSWORD = os.getenv('DB_PASSWORD') DB_NAME = os.getenv('DB_NAME') SECRET_KEY = os.getenv('SECRET_KEY', 'dev-secret-key-change-in-production') @classmethod def db_kwargs(cls): """ 返回 pymysql.connect() 需要的关键字参数字典 """ return { 'host': cls.DB_HOST, 'port': cls.DB_PORT, 'user': cls.DB_USER, 'password': cls.DB_PASSWORD, 'database': cls.DB_NAME, 'charset': 'utf8mb4', 'cursorclass': pymysql.cursors.DictCursor, } </pre>			
备注	运行的时候通过 pymysql.connect(**db_kwargs()) 连接 campus_forum 数据库, 环境发生变化时直接在 .env 上更改就行。 charset 使用 utf8mb4 以支持中文以及 emoji 显示 cursorclass 游标类型使用 DictCursor 可以使查询结果以字典的类型返回。			

3. 数据库设计 (14 分)

说明	(10 分) 按照数据表的创建顺序, 依次给出所涉及数据表的信息, 其中参照字段以“(字段 1, 字段 2, ……, 字段 n)”的形式给出, 被参照字段以“表名 (字段 1, 字段 2, ……, 字段 n)”的形式给出; (4 分) 一般 DBMS 都可以为数据库生成关系图, 请将该图片截屏并粘贴到表格中。				
数据表 (10)	创建顺序	数据表名称	主键	参照属性	被参照表及属性
	1	user	user_id	-	-
	2	section	section_id	-	-
	3	post	post_id	(user_id)	user(user_id)
				(section_id)	section(section_id)
	4	comment	comment_id	(user_id)	user(user_id)

				(post_id)	post(post_id)
				(parent_comment_id)	comment(comment_id)
	5	post_like	(user_id, post_id)	(user_id)	user(user_id)
				(post_id)	post(post_id)
	6	favorite_folder	folder_id	(user_id)	user(user_id)
	7	favorite_post	(folder_id, post_id)	(folder_id)	favorite_folder(folder_id)
				(post_id)	post(post_id)
关系图 (4)					
备注					

4. 含有事务应用的删除操作（13分）

说明	(1分) 简要说明该操作所要完成的功能； (2分) 该操作会涉及的表（必须含有两张或两张以上的关系表，同时以“表名”的形式给出） (1分) 表连接涉及字段描述（描述方式为“表1.属性=表2.属性”） (1分) 删除条件涉及的字段描述(以“表名.属性=? ”形式给出) (4分) 实现该操作的关键代码（高级语言、SQL），截图即可；（其中如果删除语句中不包含任何形式的事务应用将扣除3分） (4分) 如何执行该操作，按所述方法能够正常演示程序则给分。
功能描	删除一篇帖子，同时同步将作者用户的发帖数 post_count 减 1；该帖的评论、点赞、

述（1分）	收藏记录由外键 ON DELETE CASCADE 自动级联删除。删帖与计数更新两步必须放在同一事务中，保证 post_count 与真实帖子数始终一致。	
涉及的表（2分）	post、user	
表连接涉及字段（1分）	post.user_id = user.user_id	
删除条件字段描述（1分）	字段	规则
	post.post_id = ?	每次删除帖子时 post_count = post_count - 1
代码（4分）	<pre>def delete_with_count(post_id): """ 删除帖子，并同步减少作者的发帖计数。 两步操作放在一个事务里，保证 post_count 与真实帖子数一致。 返回 True 表示删除成功，False 表示帖子不存在。 """ from ..db import get_connection with get_connection() as conn: try: with conn.cursor() as cur: # 先查出作者是谁(删除后就查不到了) cur.execute('SELECT user_id FROM post WHERE post_id = %s', (post_id,)) row = cur.fetchone() if not row: return False # 帖子不存在 author_id = row['user_id'] # 删除帖子(评论/点赞/收藏由外键级联自动删除) cur.execute('DELETE FROM post WHERE post_id = %s', (post_id,)) # 作者发帖数 -1 cur.execute('UPDATE user SET post_count = post_count - 1 WHERE user_id = %s', (author_id,)) conn.commit() # 两步都成功，一起提交 return True except Exception: conn.rollback() # 任一步失败，全部回滚 raise</pre>	
	代码中显式使用 commit / rollback 控制事务。	

程序演示（4分）

```

○ (venv) PS D:\clDBBig\t1\src> python -u "d:\clDBBig\t1\src\admin.py"

===== 校园论坛 · 数据库管理终端 =====
1. 查看帖子总览
2. 删除帖子
3. 注销用户
4. 查看所有用户
0. 退出
=====
请选择操作：4

ID  用户名      类型      状态      发帖数
-----
1   alice       学生      正常      1
2   bob         学生      正常      1
3   charlie     学生      正常      1
4   prof_li     老师      正常      1
5   已注销用户   学生      正常      0
6   kbs         老师      正常      2

按回车继续...

===== 校园论坛 · 数据库管理终端 =====
1. 查看帖子总览
2. 删除帖子
3. 注销用户
4. 查看所有用户
0. 退出
=====
请选择操作：2

ID  标题              作者
-----
8   删除测试          kbs
7   1111              kbs
1   数据库期末怎么复习?  alice
2   三食堂二楼麻辣香锅推荐  bob
3   丢了一张校园卡      charlie
5   关于第三章的几个疑问  prof_li

输入要删除的帖子 ID: 8
帖子已删除

按回车继续...

===== 校园论坛 · 数据库管理终端 =====
1. 查看帖子总览
2. 删除帖子
3. 注销用户
4. 查看所有用户
0. 退出
=====
请选择操作：4

ID  用户名      类型      状态      发帖数
-----
1   alice       学生      正常      1
2   bob         学生      正常      1
3   charlie     学生      正常      1
4   prof_li     老师      正常      1
5   已注销用户   学生      正常      0
6   kbs         老师      正常      1

按回车继续...[]

```

使用 admin.py 删除某一用户的帖子（删除后用户 kbs 发帖数由 2 变为 1）

备注 Web 端删除逻辑一致,用户删除自己的某个帖子也会让数据库内对应的用户的发帖数-1。

5. 触发器控制下的添加操作（20 分）

说明	(1 分) 简要说明该操作所要完成的功能; (2 分) 简要说明该触发器所要完成的功能 (1 分) 该操作会涉及的表 (以 “表名” 的形式给出)。 (2 分) 该操作输入数据以及输入数据应该满足的条件, 如: 数值范围、是否为空; (6 分) 实现该操作的关键代码 (高级语言、SQL), 截图即可; (8 分) 如何执行该操作, 按所述方法能够正常演示程序则给分。	
功能描述 (1 分)	用户对帖子下的某条评论进行回复, 即向 comment 表插入一条带 parent_comment_id 的记录。	
触发器描述 (2 分)	触发器 trg_comment_same_post 在评论插入前 (BEFORE INSERT) 自动校验: 如果该评论是对另一条评论的回复, 则其所属帖子 post_id 必须与父评论的 post_id 一致, 否则通过 SIGNAL 中止插入并报错。该触发器弥补了外键无法约束跨行逻辑一致性的问题, 保证了评论层级关系的正确性。	
涉及的表 (1 分)	comment	
输入数据 (2 分)	字段	规则
	content	非空, 长度不大于 1000;
	post_id	(所属帖子 ID) 非空;
	parent_comment_id	(父评论 ID) 可空; 非空时其对应评论的 post_id 必须等于本评论的 post_id。
插入操作源码 (3 分)	合法插入: 回复帖子 1 的评论 1, 且 post_id 也为 1 INSERT INTO comment (content, user_id, post_id, parent_comment_id) VALUES ('合法的回复', 2, 1, 1); 非法插入: post_id 是帖子 2, 但回复对应的是帖子 1 的评论 1 INSERT INTO comment (content, user_id, post_id, parent_comment_id) VALUES ('非法的跨帖回复', 2, 2, 1);	

触发器源码 (3分)	<pre>-- 触发器 -- 插入评论前校验: 回复的父评论必须与本评论属于同一帖子 DELIMITER // CREATE TRIGGER trg_comment_same_post BEFORE INSERT ON comment FOR EACH ROW BEGIN DECLARE parent_post_id INT; IF NEW.parent_comment_id IS NOT NULL THEN SELECT post_id INTO parent_post_id FROM comment WHERE comment_id = NEW.parent_comment_id; IF parent_post_id <> NEW.post_id THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '回复的评论必须属于同一帖子'; END IF; END IF; END // DELIMITER ;</pre>
程序演示 (4分)	说明：向所在帖子的 post_id = 1 的父评论插入一个 post_id = 1 的子评论，不违背触发器，能够执行插入操作。 <pre>mysql> INSERT INTO comment (content, user_id, post_id, parent_comment_id) -> VALUES ('合法的回复', 2, 1, 1); Query OK, 1 row affected (0.01 sec)</pre>
程序演示 (4分)	说明：向所在帖子的 post_id = 1 的父评论插入一个 post_id = 2 的子评论，违背触发器要求，不能够执行插入操作，系统报错。 <pre>mysql> INSERT INTO comment (content, user_id, post_id, parent_comment_id) -> VALUES ('非法的跨帖回复', 2, 2, 1); ERROR 1644 (45000): 回复的评论必须属于同一帖子</pre>
备注	

6. 存储过程控制下的更新操作（18 分）

说明	（1 分）简要说明该操作所要完成的功能； （1 分）简要说明该存储过程所要完成的功能； （2 分）说明该操作涉及操作的表（必须包含两张或两张以上的关系表，以“表名形式”描述） （1 分）表连接涉及字段描述（描述方式为“表 1. 属性=表 2. 属性”） （2 分）该操作会修改字段（以“表名. 字段名”的形式给出），以及修改规则，如新数值的计算方法、在何种条件下予以修改等； （6 分）实现该操作的关键代码（高级语言、SQL），截图即可； （5 分）如何执行该操作，按所述方法能够正常演示程序则给分。	
功能描述 （1 分）	当我们需要注销一个用户时，不是直接删除 user 进行一系列的级联删除，而是将其账号下所有的帖子转移到我们预设好的系统账号“已注销用户”下面。然后还需要并同步维护双方的发帖计数，最后将该用户状态置为已注销（status=0）。	
存储过程功能描述 （1 分）	存储过程 sp_deactivate_user 接收“被注销用户 ID”和“接收方 ID”两个 IN 参数，在一个事务内完成帖子转移、双方计数更新、状态变更三步操作；通过 EXIT HANDLER 异常处理器在出错时 ROLLBACK 并 RESIGNAL，保证整个过程的原子性。 同时，还要添加约束，接收方不能等于被注销用户本身，否则 SIGNAL 报错。	
涉及的关系表 （2 分）	user、post	
表连接涉及字段(1)	post.user_id = user.user_id	
更改字段 （2 分）	字段	规则
	post.user_id	由被注销用户改为接收方 ID（帖子转移）
	user.post_count	接收方帖子数加上转移的帖子数，被注销用户清零
	user.status	被注销用户置为 0
更新代码 （3 分）	<pre> UPDATE post SET user_id = p_recipient_id WHERE user_id = p_user_id; UPDATE user SET post_count = post_count + v_post_count WHERE user_id = p_recipient_id; UPDATE user SET post_count = 0, status = 0 WHERE user_id = p_user_id; </pre>	

创建 存储 过程 源码 (3 分)	<pre>-- 存储过程 -- 注销用户: 帖子转移给系统账号 + 维护双方计数 + 用户状态改为注销 DELIMITER // CREATE PROCEDURE sp_deactivate_user(IN p_user_id INT, IN p_recipient_id INT) BEGIN -- 所有 DECLARE 必须置于 BEGIN 块最前 DECLARE v_post_count INT DEFAULT 0; DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END; IF p_user_id = p_recipient_id THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '不能将帖子转移给被注销用户自己'; END IF; START TRANSACTION; SELECT COUNT(*) INTO v_post_count FROM post WHERE user_id = p_user_id; UPDATE post SET user_id = p_recipient_id WHERE user_id = p_user_id; UPDATE user SET post_count = post_count + v_post_count WHERE user_id = p_recipient_id; UPDATE user SET post_count = 0, status = 0 WHERE user_id = p_user_id; COMMIT; END // DELIMITER ;</pre>
存储 过程 执行 源码 (1 分)	<pre>def deactivate_user(user_id, recipient_id): """调用存储过程注销用户, 转移其帖子""" with get_cursor(commit=True) as cur: cur.callproc('sp_deactivate_user', (user_id, recipient_id)) (user_repo.py)</pre>

	<pre>def deactivate_user(): """ 存储过程注销用户 """ list_users() uid = input('\n输入要注销的用户 ID: ').strip() rid = user_repo.get_system_account_id() if not uid.isdigit(): print('无效输入') return try: user_repo.deactivate_user(int(uid), int(rid)) print('注销成功, 帖子已转移') list_users() except Exception as e: print(f'! 操作被拒绝: {e}')</pre> (admin.py)
程序 演示 (2 分)	说明：将用户“kbs”注销，帖子转移至“已注销用户”，不违背存储过程，能够执行更新操作

```
O (venv) PS D:\clDBBig\t1\src> python -u "d:\clDBBig\t1\src\admin.py"
```

```
===== 校园论坛 · 数据库管理终端 =====
```

1. 查看帖子总览
2. 删除帖子
3. 注销用户
4. 查看所有用户
0. 退出

```
=====
```

请选择操作：4

ID	用户名	类型	状态	发帖数
1	alice	学生	正常	1
2	bob	学生	正常	1
3	charlie	学生	正常	1
4	prof_li	老师	正常	1
5	已注销用户	学生	正常	0
6	kbs	老师	正常	1

按回车继续...

```
===== 校园论坛 · 数据库管理终端 =====
```

1. 查看帖子总览
2. 删除帖子
3. 注销用户
4. 查看所有用户
0. 退出

```
=====
```

请选择操作：3

ID	用户名	类型	状态	发帖数
1	alice	学生	正常	1
2	bob	学生	正常	1
3	charlie	学生	正常	1
4	prof_li	老师	正常	1
5	已注销用户	学生	正常	0
6	kbs	老师	正常	1

输入要注销的用户 ID: 6

注销成功,帖子已转移

ID	用户名	类型	状态	发帖数
1	alice	学生	正常	1
2	bob	学生	正常	1
3	charlie	学生	正常	1
4	prof_li	老师	正常	1
5	已注销用户	学生	正常	1
6	kbs	老师	已注销	0

按回车继续...[]

程序演示 (2分)	说明：将“已注销用户”转移至自身，违背存储过程，系统报错； <pre>(venv) PS D:\clDBBig\t1\src> python -u "d:\clDBBig\t1\src\admin.py"</pre><pre>===== 校园论坛 · 数据库管理终端 ===== 1. 查看帖子总览 2. 删除帖子 3. 注销用户 4. 查看所有用户 0. 退出 ===== 请选择操作：4</pre><table><thead><tr><th>ID</th><th>用户名</th><th>类型</th><th>状态</th><th>发帖数</th></tr></thead><tbody><tr><td>1</td><td>alice</td><td>学生</td><td>正常</td><td>1</td></tr><tr><td>2</td><td>bob</td><td>学生</td><td>正常</td><td>1</td></tr><tr><td>3</td><td>charlie</td><td>学生</td><td>正常</td><td>1</td></tr><tr><td>4</td><td>prof_li</td><td>老师</td><td>正常</td><td>1</td></tr><tr><td>5</td><td>已注销用户</td><td>学生</td><td>正常</td><td>1</td></tr><tr><td>6</td><td>kbs</td><td>老师</td><td>已注销</td><td>0</td></tr></tbody></table><pre>按回车继续...</pre><pre>===== 校园论坛 · 数据库管理终端 ===== 1. 查看帖子总览 2. 删除帖子 3. 注销用户 4. 查看所有用户 0. 退出 ===== 请选择操作：3</pre><table><thead><tr><th>ID</th><th>用户名</th><th>类型</th><th>状态</th><th>发帖数</th></tr></thead><tbody><tr><td>1</td><td>alice</td><td>学生</td><td>正常</td><td>1</td></tr><tr><td>2</td><td>bob</td><td>学生</td><td>正常</td><td>1</td></tr><tr><td>3</td><td>charlie</td><td>学生</td><td>正常</td><td>1</td></tr><tr><td>4</td><td>prof_li</td><td>老师</td><td>正常</td><td>1</td></tr><tr><td>5</td><td>已注销用户</td><td>学生</td><td>正常</td><td>1</td></tr><tr><td>6</td><td>kbs</td><td>老师</td><td>已注销</td><td>0</td></tr></tbody></table><pre>输入要注销的用户 ID: 5 ! 操作被拒绝: (1644, '不能将帖子转移给被注销用户自己') 按回车继续...█</pre>	ID	用户名	类型	状态	发帖数	1	alice	学生	正常	1	2	bob	学生	正常	1	3	charlie	学生	正常	1	4	prof_li	老师	正常	1	5	已注销用户	学生	正常	1	6	kbs	老师	已注销	0	ID	用户名	类型	状态	发帖数	1	alice	学生	正常	1	2	bob	学生	正常	1	3	charlie	学生	正常	1	4	prof_li	老师	正常	1	5	已注销用户	学生	正常	1	6	kbs	老师	已注销	0
	ID	用户名	类型	状态	发帖数																																																																		
1	alice	学生	正常	1																																																																			
2	bob	学生	正常	1																																																																			
3	charlie	学生	正常	1																																																																			
4	prof_li	老师	正常	1																																																																			
5	已注销用户	学生	正常	1																																																																			
6	kbs	老师	已注销	0																																																																			
ID	用户名	类型	状态	发帖数																																																																			
1	alice	学生	正常	1																																																																			
2	bob	学生	正常	1																																																																			
3	charlie	学生	正常	1																																																																			
4	prof_li	老师	正常	1																																																																			
5	已注销用户	学生	正常	1																																																																			
6	kbs	老师	已注销	0																																																																			
备注	这里只展示了正常业务涉及的账户注销操作，如果要进行账号转移（一般不会涉及），也可以直接用 SQL 语句来调用存储过程。																																																																						

7. 含有视图的查询操作（15 分）

说明	（1 分）简要说明该操作所要完成的功能； （1 分）简要说明建立的该视图的功能； （2 分）简要说明该操作涉及的关系数据表（以“表名”的形式给出） （1 分）简要说明表连接涉及的字段（以“表 1. 属性=表 2. 属性”） （6 分）实现该操作的关键代码（高级语言、SQL），截图即可； （4 分）如何执行该操作，按所述方法能够正常演示程序则给分。
操作功能描述 （1 分）	查询所有帖子的总览信息，包括帖子标题、作者、所属板块、点赞数与评论数。
视图功能描述 （1 分）	创建视图 v_post_overview 将帖子表与用户表、板块表连接，并左连接点赞表、评论表进行聚合统计，把复杂的多表连接与分组查询封装成一张虚拟表，供上层直接查询。统计时使用 COUNT(DISTINCT ...) 避免多表连接时进行笛卡尔积带来的冗余行导致计数被乘算，使用 DISTINCT 统计唯一值，得到实际的计数结果。
涉及的关系表 （2 分）	post、user、section、post_like、comment
表连接字段（1 分）	post.user_id = user.user_id post.section_id = section.section_id post.post_id = post_like.post_id post.post_id = comment.post_id

创建
视图
代码
(3
分)

-- 视图

```
CREATE VIEW v_post_overview AS
SELECT
    p.post_id,
    p.title,
    p.publish_time,
    u.user_id AS author_id,
    u.username AS author,
    s.section_id,
    s.section_name,
    COUNT(DISTINCT pl.user_id) AS like_count,
    COUNT(DISTINCT c.comment_id) AS comment_count
FROM post p
JOIN user u ON p.user_id = u.user_id
JOIN section s ON p.section_id = s.section_id
LEFT JOIN post_like pl ON p.post_id = pl.post_id
LEFT JOIN comment c ON p.post_id = c.post_id
GROUP BY p.post_id, p.title, p.publish_time,
         u.user_id, u.username, s.section_id, s.section_name;
```

查询
代码
(3
分)

```
def list_from_view(section_name=None):
    """通过视图查询帖子总览"""
    sql = 'SELECT * FROM v_post_overview'
    params = []
    if section_name:
        sql += ' WHERE section_name = %s'
        params.append(section_name)
    sql += ' ORDER BY like_count DESC'
    with get_cursor() as cur:
        cur.execute(sql, params)
        return cur.fetchall()
```

(post_repo.py)

程序
演示
(4
分)

```
○ (venv) PS D:\clDBBig\t1\src> python -u "d:\clDBBig\t1\src\admin.py"

===== 校园论坛 · 数据库管理终端 =====
  1. 查看帖子总览
  2. 删除帖子
  3. 注销用户
  4. 查看所有用户
  0. 退出

=====
请选择操作：1

=== 帖子总览 ===
ID  标题                                     作者      板块      赞      评论
-----
1   数据库期末怎么复习?                    alice     学习交流      3      5
2   三食堂二楼麻辣香锅推荐                bob       校园生活      2      2
5   关于第三章的几个疑问                  prof_li    学习交流      2      0
3   丢了一张校园卡                        charlie    失物招领      0      2
7   1111                                    已注销用户 学习交流      0      0

按回车继续...[]
```

或者直接在 MySQL 中查询：

```
mysql> SELECT * FROM v_post_overview
-> ORDER BY like_count DESC;

+-----+-----+-----+-----+-----+-----+-----+-----+
| post_id | title                                     | publish_time | author_id | author | section_id | section_name | like_count | comment_count |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 数据库期末怎么复习? | 2026-06-03 09:07:40 | 1 | alice | 1 | 学习交流 | 3 | 5 |
| 2 | 三食堂二楼麻辣香锅推荐 | 2026-06-03 09:07:40 | 2 | bob | 2 | 校园生活 | 2 | 2 |
| 5 | 关于第三章的几个疑问 | 2026-06-03 09:07:40 | 4 | prof_li | 1 | 学习交流 | 2 | 0 |
| 3 | 丢了一张校园卡 | 2026-06-03 09:07:40 | 3 | charlie | 3 | 失物招领 | 0 | 2 |
| 7 | 1111 | 2026-06-05 11:26:24 | 5 | 已注销用户 | 1 | 学习交流 | 0 | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

备注